Visualizing Functions and Data with Gnuplot

Roychowdhury & Banerjee

Syllabus

Basic 2D and 3D graph plotting, plotting functions, polar and parametric plots, modifying graph appearance (colors, line styles, labels), surface and contour plots, logarithmic scaling, grid lines, multiplot layouts, and exporting plots to various formats.

©Shibaji Banerjee and Suparna Roychowdhury: All rights for circulation, modification, utilization and (re)distribution of this document or parts thereof lies solely with the copyright holders and authors of this document.

1 Basic 2D Plotting

Welcome to your journey in Computational Physics! This course introduces tools to visualize and analyze functions, starting with *gnuplot* , a powerful, command-line driven plotting tool. We begin this journey by learning to visualize elementary functions and analyze them through *gnuplot* , which would help in developing an intuitive understanding of different aspects of functions, data, and the information they contain. In particular, even if rigorous model building might be one's ultimate goal, graphical methods still need to be the first step, so that a sense for the data, its behavior, and quality is developed. Visualizing data and fitting models to it are crucial in Computational Physics, enabling researchers to identify patterns, test hypotheses, and validate theoretical models against experimental results. *gnuplot* helps you create both quick exploratory graphs and publication-quality visuals, fostering an intuitive understanding of mathematical functions and their properties. As you would realize later, this would prove to be an extremely important gadget which you can pick and safe-keep for your future endeavors in Physics or any form of scientific pursuit.

gnuplot generates plots from functions, offering interactive output in a graphical window. You issue commands at the `gnuplot>` prompt, and the plot updates instantly. This section focuses on plotting elementary functions, customizing ranges, and styling graphs for better visualization, laying the foundation for data analysis techniques introduced later in the course.

1.1 Invoking *gnuplot*

To start *gnuplot* , open a terminal and type:

```
gnuplot
```

This opens a *gnuplot* window displaying a welcome message and the prompt:

```
gnuplot>
```

Enter commands here to generate plots. To exit, type `exit`, `quit`, or press `Ctrl+D`. For example, to plot a sine function:

```
gnuplot> plot sin(x)
```

gnuplot samples 100 points over the default x-range $[-10, 10]$. You can adjust the range, sampling rate, and appearance to suit your needs. The following **Facts** section lists key commands. Experiment with these to build familiarity with *gnuplot* .

1.2 Facts: Essential *gnuplot* Commands for Function Plotting

1. **Plotting a Single Function**: To plot a function $f(x)$, use:

   ```
   gnuplot> plot f(x)
   ```

   Define functions beforehand if needed, e.g.:

   ```
   gnuplot> f(x) = sin(x) + cos(x)*exp(-x)
   gnuplot> plot f(x)
   ```

   Note: Use `**` for exponents (e.g., `x**2` for x^2). Ensure multiplication is explicit (e.g., `2*x`, not `2x`).

2. **Plotting Multiple Functions**: Plot multiple functions in one command, separated by commas:

   ```
   gnuplot> plot sin(x), cos(x)
   ```

3. **Setting Axis Ranges**: Customize x- and y-ranges with:

©Shibaji Banerjee and Suparna Roychowdhury, See Page 1 for full text of copyright notice

```
gnuplot> set xrange [-3:3]
gnuplot> set yrange [0:10]
```

The default x-range is $[-10, 10]$; y-range adjusts automatically.

4. **Displaying Axes**: Show x=0 and y=0 axes with:

```
gnuplot> set zeroaxis
```

5. **Adjusting Sampling Rate**: Increase samples for smoother curves, especially for sharp features:

```
gnuplot> set samples 1000
```

See Exercise 1(c) for an example with discontinuities. **Example**: For a function with sharp features like $f(x) = \tan(x)$, the default sampling (100 points) may miss asymptotes or appear discontinuous. Try:

```
gnuplot> set xrange [0:pi]
gnuplot> set samples 100
gnuplot> plot sin(100*x)
```

This may show jagged lines. Increase sampling:

```
gnuplot> set samples 1000
gnuplot> replot
```

Higher sampling reveals the steep behavior near asymptotes more clearly, avoiding artifacts that resemble discontinuities.

6. **Customizing Line Styles**: Adjust line width (`lw`) and color (`lc`):

```
gnuplot> plot sin(x) lw 2 lc 'red', cos(x) lw 1 lc 'blue'
```

7. **Setting Plot Titles**: Override default legend labels:

```
gnuplot> plot sin(x) lw 2 title 'The Sinusoid'
```

Options like `lw`, `lc`, and `title` are space-separated; traces are comma-separated.

8. **Plotting Data Points on Functions**: Define points in a data block and plot them:

```
gnuplot> $data << EOD
1.57 1.00
7.85 1.00
EOD
gnuplot> set xrange [0:10]
gnuplot> plot sin(x) lw 2, '$data' with points ps 4
```

©Shibaji Banerjee and Suparna Roychowdhury, See Page 1 for full text of copyright notice

The `ps` option sets point size. Data blocks end with `EOD`. Use this to mark extrema or intercepts. **Note**: The variable `$data` is a rather special variable; ordinary variables without the dollar prefix (like `a`, `b`, `c`, `ndata`, etc.) can be defined and used in *gnuplot* with a numerical or string data-value (e.g., `a=4` followed by `f(x)=x**a`) just like variables in other computer languages. Some variables like `pi` are already defined. You can define and use others, e.g., `e=exp(1)` followed by `print e` which displays its value on the terminal.

9. **Multiple Plot Windows**: Open multiple windows, but only the last is active:

```
gnuplot> set term win enh 0
gnuplot> test
gnuplot> set term wxt enh 1
gnuplot> plot sin(x)
```

   The `test` command displays terminal properties.

10. **Using Grid Lines**: Add a grid for better readability and estimation:

```
gnuplot> set grid
```

11. **Logarithmic Scaling**: Use logarithmic axes for exponential or power-law functions:

```
gnuplot> set logscale y
gnuplot> plot exp(x)
```

    Reset with `unset logscale y`. Useful for visualizing wide dynamic ranges.

12. **Saving and Loading Scripts**: Save commands in a text file (e.g., `myplot.plt`) and load it:

```
gnuplot> load 'myplot.plt'
```

    Split long lines with a backslash (\):

```
plot sin(x) lw 3, \
     cos(x) lw 1
```

    Note: Writing down the commands one after another in a text file facilitates 'loading' this file each time you decide to change something about the plot by just changing the relevant command in the file. This is the **preferred way** to run *gnuplot* , as it encourages both graphical experiments and generation of publication-quality graphs with a high degree of finish. Assuming such a file (`myplot.plt`) has been created and is present in the current working directory, it can be loaded (repeatedly) to regenerate or modify the plot.

13. **Resetting Settings**: Reset *gnuplot* to default settings:

```
gnuplot> reset
```

14. **Exporting Plots**: Save plots as PNG or other formats:

```
gnuplot> set terminal pngcairo
gnuplot> set output 'plot.png'
gnuplot> plot sin(x)
gnuplot> set output
gnuplot> set terminal win
```

©Shibaji Banerjee and Suparna Roychowdhury, See Page 1 for full text of copyright notice

1.3 Tips and Tricks for Function Visualization

Here are techniques to enhance your understanding and visualization of functions in *gnuplot* , combining interactive graph inspection tools and advanced plotting methods:

1. **Manual Zooming**: To zoom on a part of a graph, change its axis limits:

   ```
   gnuplot> set xrange [0:2]
   gnuplot> set yrange [-1:1]
   gnuplot> replot
   ```

   This allows precise focus on specific function features.

2. **Mouse Wheel Zooming**: Place the mouse cursor on the curve, hold `Ctrl`, and roll the mouse wheel to zoom in/out. This works for both `win` and `wxt` terminals. Press `u` to return to the original, un-zoomed view. Useful for inspecting local maxima or minima.

3. **Zoom Rectangle**: Drag with the right mouse button (RMB) to draw a zoom rectangle, then click RMB to zoom into that region. This is ideal for examining specific intervals of a function.

4. **Panning**: Use the mouse wheel to pan in the y-direction or Shift + wheel to pan in the x-direction. This helps adjust the view without resetting ranges.

5. **Copying Coordinates**: Double-click on a point to copy its coordinates to the clipboard. Paste these into the *gnuplot* workspace or a text file for further analysis, e.g., to mark critical points or intersections.

6. **Marking Points with Labels**: Press the mouse wheel on a point to mark it with its coordinates. These labels are temporary and lost on zooming. For permanent labels, use:

   ```
   gnuplot> set mouse labels
   ```

   Then press the wheel to place labels. Remove permanent labels by holding `Ctrl` and pressing the wheel on the same point. Useful for annotating zeros or extrema.

7. **Using Rulers**: Press `r` over a point to display a vertical ruler. Press `r` again to remove it. While active, the ruler shows the distance in user units from the ruler's origin to the mouse cursor, aiding in measuring function properties like periods or amplitudes.

8. **Adding Labels and Annotations**: Place text labels or arrows for key features:

   ```
   gnuplot> set label 'Maximum' at 1.57,1.0
   gnuplot> set arrow from 1.57,0 to 1.57,1.0 nohead
   ```

   Use this to highlight zeros, maxima, or asymptotes.

9. **Customizing Legends**: Position and style the key (legend):

   ```
   gnuplot> set key outside right top
   gnuplot> set key box
   ```

   Helpful for comparing multiple functions without cluttering the plot.

10. **Plotting Derivatives**: Visualize function behavior by plotting derivatives:

©Shibaji Banerjee and Suparna Roychowdhury, See Page 1 for full text of copyright notice

```
gnuplot> f(x) = sin(x)
gnuplot> df(x) = cos(x)  # Approximate derivative or use symbolic if needed
gnuplot> plot f(x) title 'f(x)', df(x) title "f'(x)"
```

This reveals slopes, concavity, and critical points.

11. **Using Variables for Parametric Exploration**: Define parameters to vary functions:

```
gnuplot> a = 1.0
gnuplot> f(x) = a * sin(x)
gnuplot> plot f(x)
gnuplot> a = 2.0; replot
```

The `replot` command updates the plot without retyping.

12. **Avoiding Common Pitfalls**: - For discontinuities or asymptotes, increase samples and set appropriate ranges to avoid artifacts. - Use `set autoscale` to let *gnuplot* choose ranges, but override for focus. - Check function definitions for missing operators (e.g., `*` for multiplication).

©Shibaji Banerjee and Suparna Roychowdhury, See Page 1 for full text of copyright notice

Exercise 1 For each part, follow these steps: 1. Analyze the function mathematically (e.g., find zeros, symmetries, asymptotes). 2. Sketch the graph by hand based on your analysis. 3. Use *gnuplot* to plot the function and verify your sketch. 4. Include the specified customizations (ranges, styles, etc.).

(a) **Function**: $f(x) = x + \sin(x)$, $x \in (-2\pi, 2\pi)$
Instructions: Explain how this function is a translation of $\sin(x)$ by adding x. Plot $f(x)$ and $y = x$ on the same graph to show this. Set `lw 2` for $f(x)$ and `lw 1` for $y = x$. Title $f(x)$ as "Translated Sine" and $y = x$ as "Linear Component". Use `set grid`.

(b) **Function**: $f(x) = x^2 + x + 1$, $x \in (-3, 3)$
Instructions: Rewrite $f(x)$ as a completed square to find the minimum. Identify the y-intercept. Plot the function with `lw 2`, titled "Function". Mark the y-intercept $(0, 1)$ and minimum with points (`pt 7` for y-intercept, `pt 5` for minimum, `ps 3` for both). Title points as "Y-intercept" and "Minima". Show the y=0 axis with `set zeroaxis`. Add a label at the minimum.

(c) **Function**: $f(x) = x(x^2 - 1)$
Instructions: Find and mark the roots. Use `lw 2` and `set zeroaxis`. Plot points with `ps 3` and title them "Roots". Add arrows pointing to the roots.

(d) **Function**: $f(x) = \frac{x^2}{x-1}$, $y \in (-10, 10)$
Instructions: Set `set samples 400` for smooth plotting near the discontinuity at $x = 1$. Show the vertical asymptote at $x = 1$ using:

```
gnuplot> set arrow from 1,-10 to 1,10 nohead lc 'black'
```

Hint: To find the oblique asymptote, perform polynomial division on x^2 by $x - 1$. Divide the leading term x^2 by x, multiply the result by $x - 1$, subtract, and continue with the remainder to find a linear quotient. Plot this linear function as the oblique asymptote on the same graph. Use `set zeroaxis` and `set grid`.

(e) **Function**: $f(x) = (ax)^2 e^{-ax}$, $x \in (0, 10)$, $a = 1, 2, 3$
Instructions: Plot all three curves ($a = 1, 2, 3$) on one graph, colored red, green, and blue, respectively. Find approximate maxima by double-clicking each curve in *gnuplot* to copy coordinates. Define these points in a data block and plot them with `ps 4`. Draw vertical arrows from each maximum to $y = 0$ using `set arrow`. Title curves as "a=1", "a=2", "a=3". Position the key outside.

Exercise 2 Use *gnuplot* to extract information from graphs. For each part: 1. Plot the function(s) with specified settings. 2. Use mouse interactions (zoom, rulers, clicks) to estimate key features. 3. Verify estimates analytically where instructed.

(a) **Function**: $f(x) = x^3 + 1 - \exp(x)$, $x \in [-8, 8]$
Instructions: Set `zeroaxis` and `yrange [-5:5]` to show all four x-intercepts. Use the mouse to estimate their coordinates. Verify by solving $x^3 + 1 = \exp(x)$ numerically (e.g., using *gnuplot* 's `print` or a calculator). Add grid and labels for intercepts.

(b) **Function**: $f(x) = 3x^4 - 6x^2$, $x \in [-10, 10]$
Instructions: Plot with automatic y-scaling. Zoom in to clearly see x-intercepts using `set xrange [-2:2]` and `set yrange [-5:5]`. Estimate intercepts with the mouse. Verify roots by solving $3x^4 - 6x^2 = 0$. Use logarithmic y-scale if helpful for large ranges.

(c) **Functions**: $g(x) = 5\exp(-0.5x)$, $h(x) = x + 1$
Instructions: Define and plot both functions. Experiment with `xrange` and `yrange` to find their intersection. Estimate the intersection point by left-clicking. Verify by solving $5\exp(-0.5x) = x + 1$ using `print` or a calculator. Add an arrow to the intersection.

(d) **Function**: $k(x) = x + 3\sin(2x)$, $x \in [-1, 8]$
Instructions: Plot $k(x)$ and the line $y = 4$. Estimate x-values where $k(x) = 4$ using the mouse. Use `set grid` for better accuracy. Count and report approximate intersection points. Plot the derivative to visualize slopes.

©Shibaji Banerjee and Suparna Roychowdhury, See Page 1 for full text of copyright notice

(e) **Function**: $f(x) = \exp(-x/100)\cos(x),\ x \in [0, 100]$
Instructions: Plot with `set samples 1000` for a smooth curve. Experiment with `set samples 100, 500, 2000` to observe smoothness. Use `set grid` and zoom to examine oscillatory behavior. Try logarithmic y-scale to see envelope.

©Shibaji Banerjee and Suparna Roychowdhury, See Page 1 for full text of copyright notice